موضوع الدرس : ..... اليوم : .....
التاريخ : / / ٢٠ م الموافق : / / ١٤ هـ

Project Analysis: Github Trending API and Debian Package

1. file Organization

The project's files are structured to mimic an Ubuntu/Debian file system, which is essential for building the .deb package.

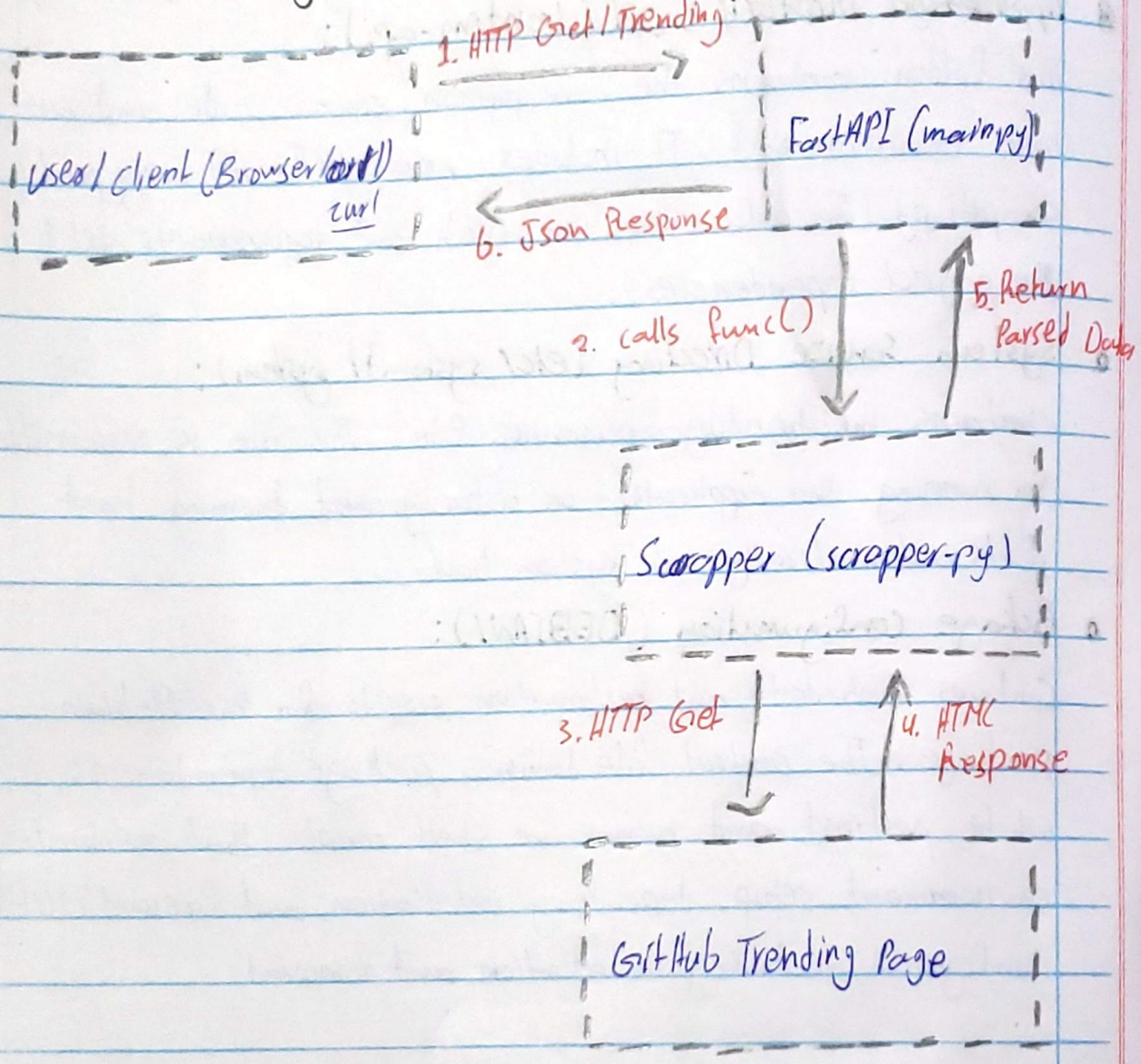
- **Application Directory (opt/trending-api):**
  This folder contains the core python source code and run-time environment. It includes main.py (The API entry point), scraper.py (The data extraction logic), and requirements.txt (The project dependencies).
- **System Service Directory (etc/systemd/system):**
  Contains the trending-api.service file. This file is responsible for running the application as a background daemon that starts automatically on system boot.
- **Package Configuration (DEBIAN/):**
  Contains metadata and automation scripts for the Debian package. The control file defines package dependencies, while postinst and prerm are shell scripts that automate environment setup, dependency installation, and firewall (UFW) configuration during installation and removal.

موضوع الدرس :
التاريخ : / / ٢٠م
اليوم :
الموافق : / / ١٤هـ

2. Data Flow

The data flows seamlessly from the user's request to Github and back as a JSON response.

- Data Flow Diagram



STAR

الواجب :

موضوع الدرس : ..... اليوم : .....
التاريخ : / / ٢٠ م الموافق : / / ١٤ هـ

- steps Explanation :

1. Request: The user sends an HTTP Get request to http://local host:8000/trending.
2. Routing: FastAPI receives the request in main.py and calls the fetch-trending-repos() function.
3. Fetching: The scraper.py script sends an HTTP GET request to GitHub's trending pag.
4. HTML Response: GitHub returns the raw HTML of the page.
5. Parsing: The scraper uses BeautifulSoup to extract the repository name, description, and language, then formats it into a list of dictionaries and sends it back to the API.
6. Response: FastAPI converts the Python dictionary into a JSON object and sends it back to the user.

3. Why this design was chosen?

This system was designed using the "Separation of concerns" principle to ensure maintainability and professional deployment:

- **Separated Scraping Layer:**
  Isolating the web scraping logic in scraper.py ensures that if GitHub changes its HTML structure, we only need to update one file without touching the API logic.

موضوع الدرس :
التاريخ : / / ٢٠ م
اليوم :
الموافق : / / ١٤ هـ

- **Separated API layer:**
  Using FastAPI provides a fast, modern network interface with automatic Swagger documentation (/docs) for easy testing.
- **OS Management:**
  Relying on systemd ensures high reliability, as the operating system will manage the process and automatically restart it if it crashes.
- **Automated Deployment:**
  Wrapping the entire project in a .deb package makes deployment "zero-touch". The post-installation scripts automatically configure the Python virtual environment and firewall, abstracting away complex server setups from the end-user.

STAR

الواجب :